PROJETO DE CURSO • ETAPA 2

Documento de Arquitetura e Projeto

Sistema de Gerenciamento de Serviços de Manutenção

Arquitetura revisada — versão 0.2 | evidências confrontadas com o repositório público em 12/11/2026

Identificação	Informação
Disciplina	CSI410 — Engenharia de Software II | UFOP / ICEA
Estudante	Larissa Kethen Gonçalves Oliveira | Matrícula: 22.2.8132
Modalidade	Projeto de curso individual
Cliente principal	Geraldo Palmeira Brasil — profissional autônomo de manutenção em eletrônicos
Emissão	12 de novembro de 2026 | Entrega inicial prevista: 15/11/2026
Situação	Esqueleto Django publicado; 2 commits e evidência de 10 testes aprovados. Funcionalidades futuras ainda não implementadas.

Nota de integridade documental: as evidências de código, commits e testes são verificáveis no GitHub. A validação de estados, capacidade, critérios automáticos e métricas não foi comprovada; permanecem hipóteses e próximos incrementos.

Sumário executivo

Esta etapa transforma os requisitos da Etapa 1 em decisões de arquitetura, fronteiras de módulos, padrões de projeto e um esqueleto executável. A versão revisada incorpora a implementação publicada e a execução documentada de 10 testes, distinguindo evidências reais de funcionalidades planejadas. O protótipo é local e acadêmico; não configura um produto pronto para uso pelo cliente.

Base implementada no esqueleto: monólito modular Django (requirements.txt: Django==6.1.3), SQLite via ORM, templates HTML e CSS responsivo. A versão exata do Python e do SQLite utilizados na máquina não foi registrada nas evidências.

1. Contexto e requisitos de arquitetura

Problema confirmado. O profissional atua sozinho, registra nome do cliente e serviço em anotações, considera urgência/risco e tempo de execução para priorizar demandas e recusa algumas solicitações quando há sobrecarga. Foi discutida a necessidade de organizar atendimentos e acompanhar prazos. O protótipo atual implementa somente cadastro e ordenação por prioridade manual provisória.

Hipóteses a validar com o cliente. Fluxo de estados (solicitado, orçado, aprovado, em execução, concluído), capacidade diária, estimativas e alertas de conflito foram recomendados para investigação; não há cálculo ou transições automáticas implementados no código publicado.

ID	Requisito/cenário	Implicação arquitetural
RN01 / H02-H03	Prioridade por risco e tempo; ordenação de serviços	Política isolada de priorização e consulta ordenada.
RN02 / H06-H07	Capacidade limitada; prevenção de sobrecarga	Planejamento de carga como módulo independente; alertar, não rejeitar automaticamente sem validação.
RN03 / H04	Prazos de entrega	Persistir previsão de conclusão e permitir filtros/consulta.
RN04 / H05	Estados do serviço (hipótese docente)	Definir transições em um ponto controlado; evitar alterações diretas sem validação.
C01	Usabilidade no celular (meta proposta: até 60 s / 5 passos)	Interface responsiva, formulários curtos e medição com usuário.
C02	Modificabilidade (meta proposta: até 4 h para mudança simples)	Segregar regras de prioridade das views e da persistência.
C03	Desempenho (meta proposta: 100 registros / até 2 s)	Listagem paginável, consultas eficientes e teste medido.

Escopo mínimo efetivamente demonstrado: H01 (cadastro básico) e suporte provisório a H02-H03 por seleção manual de nível e listagem ordenada. A regra automática por risco/tempo, H04-H08 e as metas de qualidade dependem de evolução e validação com o cliente.

2. Visão da arquitetura proposta

A solução foi organizada como aplicação Django de execução local, em uma unidade lógica (monólito modular). O navegador acessa as views; o sistema utiliza casos de uso, Strategy para ordenar prioridades manuais e Adapter para converter objetos de domínio em registros do Django ORM. O SQLite armazena somente dados fictícios. Não há integração com WhatsApp, pagamentos ou hospedagem em produção.

2.1 Contexto e contêineres

Leitura do diagrama: a figura representa a arquitetura-alvo do protótipo. O profissional acessa o sistema pelo navegador do celular ou computador; clientes atendidos não têm contas no escopo atual. As telas HTML invocam a aplicação Django, que persiste dados via ORM/SQLite. A adequação de uso no celular ainda não foi cronometrada.

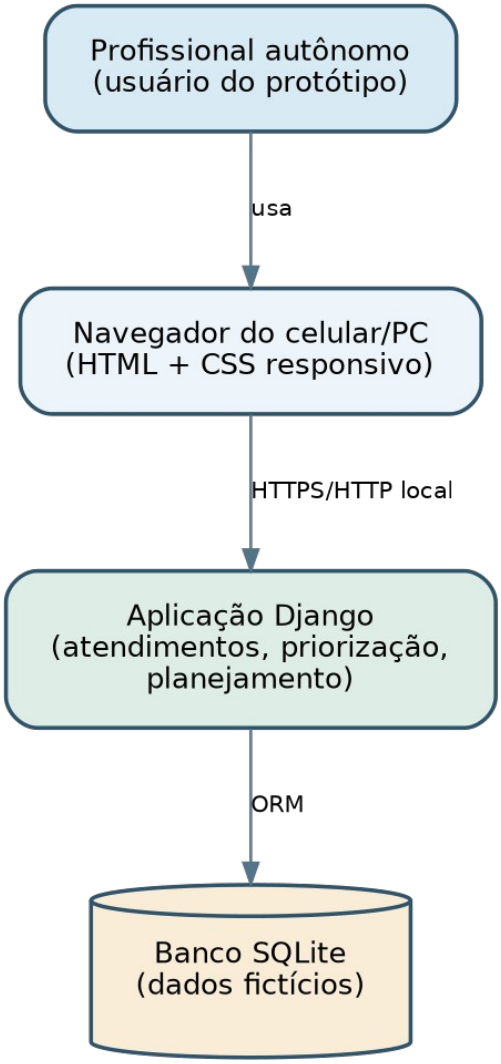


Figura 1 — Contexto e contêineres da solução proposta.

2.2 Módulos e fluxo de chamadas

No fluxo implementado, views e formulários chamam ServicoAtendimentos por meio de um ponto de composição, com PoliticaPrioridade e RepositorioAtendimento injetados. A persistência é feita pelo DjangoAtendimentoAdapter. O módulo planejamento existe como interface de evolução: não calcula capacidade, conflitos nem prazos.

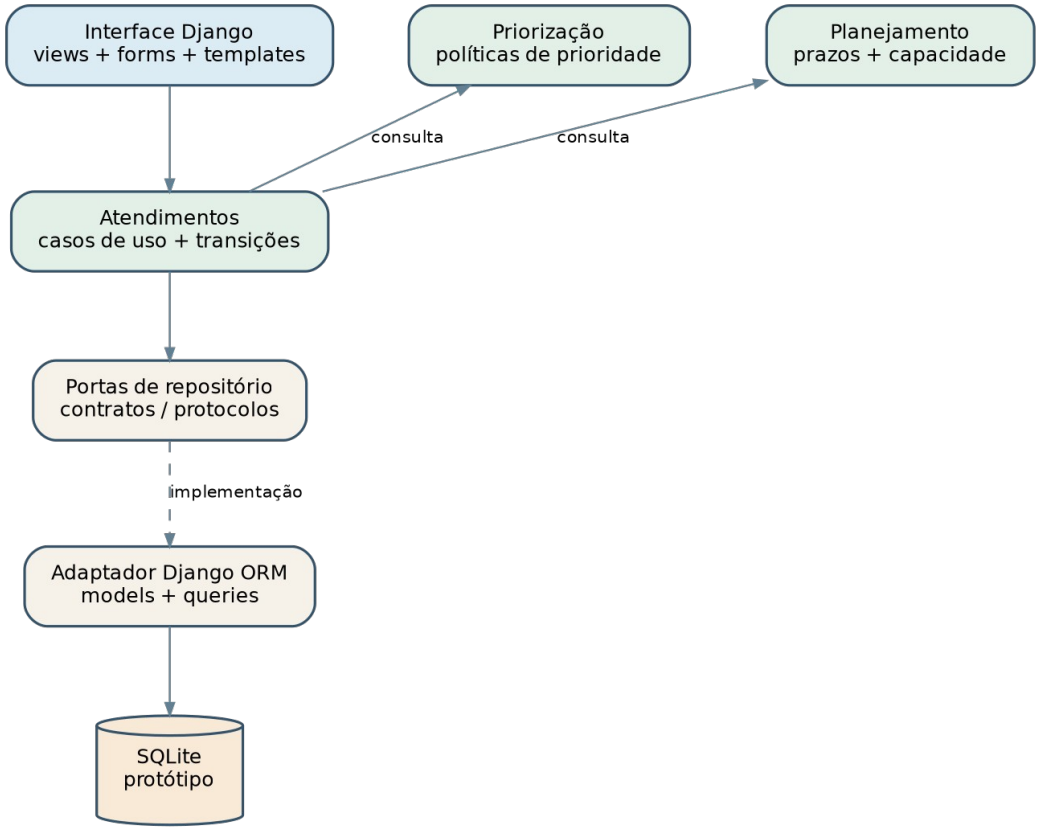


Figura 2 — Decomposição lógica e principais dependências.

Regra geral: as setas descrevem chamadas permitidas; não significam importação circular. A separação é lógica, dentro de um mesmo processo Django. O detalhe das dependências aparece no Template 15 (Seção 4).

2.3 Fluxo de estados de serviço — hipótese para validação

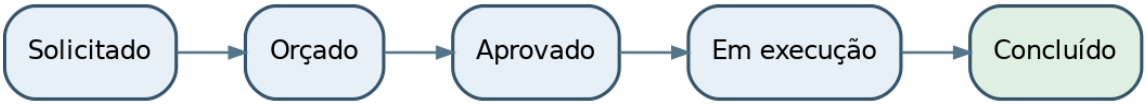


Figura 3 — Fluxo sugerido pelo professor; transições e exceções dependem do cliente.

Somente o estado inicial SOLICITADO é atribuído ao cadastro atual. As etapas do fluxo da Figura 3 representam hipótese do professor, não funcionalidades implementadas. Recusas, cancelamentos, reaberturas e transições permitidas dependem de validação com o cliente.

2.4 Exemplo de fluxo de registro



Figura 4 — Fluxo conceitual do cadastro e consulta à política de prioridade.

3. Registros de decisão de arquitetura — Template 14

ADRs 001 a 005: decisões materializadas no esqueleto e verificadas por inspeção do código, ainda sem ratificação formal registrada. ADR-006: proposta de evolução, dependente de validação com o cliente. As justificativas completas estão neste documento; no repositório docs/adr/ há apenas um README de síntese (não existem ainda seis arquivos individuais de ADR).

ADR-001 — Stack de desenvolvimento

Status: IMPLEMENTADO NO ESQUELETO; formalização da aceitação pendente | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: C01, C02, C03; H01-H04

Contexto. O protótipo é individual, orientado a formulários e regras de negócio, e tem prazo acadêmico curto. A estudante ainda está aprendendo Python/Django, o que aumenta o risco de curva de aprendizado.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Python + Django + SQLite	Aplicação web integrada; ORM, templates, formulários e testes nativos; banco local simples.	Aprendizado inicial de Python/Django; SQLite é menos indicado a alta concorrência.
Java + Spring Boot + PostgreSQL	Ecossistema de testes e separação de responsabilidades robustos.	Mais configuração no início; necessidade de infraestrutura para banco.
Node.js + Express + React + PostgreSQL	Frontend separado e interfaces interativas.	Mais camadas, contratos de API, dependências e esforço de integração.

Decisão adotada no esqueleto. Usar Django Templates, Django ORM e SQLite, com Django==6.1.3 declarado em requirements.txt. O ambiente de execução local e os testes constam das evidências, mas a versão exata do interpretador Python não foi comprovada por log versionado.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Menos infraestrutura e instalação simples.	Curva de aprendizado e limites de concorrência do SQLite.	Migrar de framework implica refazer interface e camadas; o ORM reduz parte do esforço de mudança de banco.

Evidências disponíveis. Há arquivos de projeto Django, modelos, migração inicial, telas e 10 testes com resultado OK em docs/evidencias/resultado_testes.txt. A estudante relatou execução local da interface. Capturas adicionais de tela e versões do Python/SQLite podem complementar a entrega; não são afirmadas como anexadas.

ADR-002 — Estilo arquitetural

Status: IMPLEMENTADO NO ESQUELETO; formalização da aceitação pendente | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: RN01-RN05; C02

Contexto. As regras de prioridade, estados e capacidade podem mudar com novas decisões do cliente. O sistema tem um operador e não exige escalabilidade independente de subsistemas.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Monólito sem separação por domínio	Menor quantidade de arquivos e configuração inicial.	Regras espalhadas pelas views/models, acoplamento elevado.
Monólito modular	Um deploy único, com fronteiras internas e testes isolados.	Exige manter contratos e regras de dependência consistentes.
Microsserviços	Implantação e escala por serviço.	Custo operacional, comunicação de rede e testes distribuídos desproporcionais ao escopo.

Decisão implementada. A aplicação foi separada em apps/pacotes de interface_web, atendimentos, priorizacao, planejamento e persistencia. Views chamam casos de uso; a lógica da prioridade manual não depende do ORM. O módulo de planejamento ainda é um esqueleto sem cálculos.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Facilita a localização das alterações e evita infraestrutura distribuída.	Mais disciplina e alguns arquivos de contrato.	Dividir em serviços independentes mais tarde exigirá repensar transações e comunicação.

Evidência e limites. O código mostra separação de responsabilidades e testes de serviço sem banco. Não existe registro de uma alteração de política cronometrada, nem análise automatizada de ciclos de importação ou comprovação da meta de modificabilidade de 4 horas.

ADR-003 — Interface e interação móvel

Status: IMPLEMENTADO NO ESQUELETO; formalização da aceitação pendente | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: C01; H01-H04

Contexto. O usuário precisa realizar tarefas rápidas pelo celular. Não há necessidade confirmada de aplicativo nativo, trabalho offline ou frontend desacoplado.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Django Templates + HTML/CSS responsivo	Sem segundo projeto nem API obrigatória; formulários simples.	Atualizações podem depender de novas requisições; menor dinamismo do lado cliente.
React SPA + API REST Django	Interatividade e separação de clientes.	Aumenta esforço de frontend, autenticação, estado, integração e testes.

Decisão implementada na interface inicial. O protótipo usa Django Templates, formulários Django e CSS responsivo para a página inicial e o cadastro de atendimento fictício. Não há SPA nem API externa para o frontend.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Reduz esforço e favorece entregas rápidas.	Menor flexibilidade para interações avançadas no futuro.	Adotar SPA depois exige criar API e revisar organização do frontend.

Evidência e pendência. Testes de integração verificam o carregamento da página, o cadastro válido e a recusa de formulário inválido. O ensaio com cliente em dispositivo móvel e a meta de até 60 segundos ou 5 passos ainda não foram medidos.

ADR-004 — Política de priorização

Status: IMPLEMENTADO NO ESQUELETO; formalização da aceitação pendente | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: RN01, H02-H03, C02

Contexto. O cliente considera risco e duração, mas ainda não informou pesos, níveis fixos ou critério de desempate. A aplicação não deve inventar uma pontuação automática como se fosse regra validada.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Prioridade manual diretamente na view	Entrega inicial rápida.	Regra acoplada à interface e difícil de substituir por novos critérios.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Política encapsulada (Strategy)	Permite trocar política manual por política baseada em critérios depois, sem alterar interface.	Adiciona protocolo e classes; pode ser excesso se nunca houver nova política.

Decisão implementada provisoriamente. A política PrioridadeManual aplica a ordem ALTA, MEDIA e BAIXA, com desempate pelo menor ID. O contrato PoliticaPrioridade expõe chave_ordenacao(), consumida por ServicoDePriorizacao. Os níveis e o desempate não foram confirmados pelo cliente; não existe pontuação automática por risco ou duração.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Regras ficam testáveis e substituíveis.	Pequena sobrecarga de abstração para a versão inicial.	Alterar contrato público da política afeta os consumidores; mudança de semântica dos níveis pode exigir migração de dados.

Evidência e limites. tests/test_priorizacao.py verifica ordenação das três prioridades e desempate por ID. A possibilidade estrutural de substituir a Strategy existe, mas não há teste publicado de troca entre duas implementações diferentes ou de edição de prioridade existente.

ADR-005 — Persistência e isolamento de dados

Status: IMPLEMENTADO NO ESQUELETO; formalização da aceitação pendente | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: RN01-RN03; C03

Contexto. O protótipo usa dados fictícios e terá carga pequena, sem infraestrutura de produção. O esquema pode mudar durante o projeto.

Alternativa	Vantagens no projeto	Desvantagens no projeto
SQLite via Django ORM	Banco local e sem servidor; migrations integradas.	Limitações para concorrência de gravação e cenários de produção.
PostgreSQL via Django ORM	Maior aderência a implantação multiusuário.	Exige serviço de banco e mais configuração para demonstrar localmente.

Decisão implementada no protótipo. Os atendimentos fictícios são armazenados por Django ORM em SQLite. A porta RepositorioAtendimento fica em apps/atendimentos/ports.py, e DjangoAtendimentoAdapter implementa salvar() e listar(), convertendo registros entre ORM e domínio.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Instalação e demonstração simples; evolução de esquema versionável.	Limites do SQLite em concorrência e implantação.	Recursos específicos do SQLite e queries acopladas dificultariam troca de banco; evitá-los.

Evidência e limites. A migração persistencia.0001_initial e o teste de integração test_salva_e_recupera_atendimento_via_adapter estão no repositório; o log registra a execução de migrations e 10 testes aprovados. Testes específicos de concorrência, desempenho ou transações complexas não constam das evidências.

ADR-006 — Tratamento de capacidade e conflitos

Status: PROPOSTO, NÃO IMPLEMENTADO — depende de validação | Responsável: Larissa (projeto individual) | Referência: 12/11/2026

Regras / cenários relacionados: RN02, RN04, RN05; H05-H07

Contexto. O professor pediu investigação sobre capacidade por dia, estados e alertas. O cliente relatou recusar serviços em excesso, mas não forneceu uma fórmula de duração, horas por dia nem regras de bloqueio.

Alternativa	Vantagens no projeto	Desvantagens no projeto
Bloqueio automático de agendamento	Impede sobrecarga segundo algoritmo.	Pode rejeitar serviços de forma inadequada se estimativas e limites não forem confirmados.
Alerta e decisão humana	Preserva a decisão do profissional; funciona com estimativas inicialmente incertas.	Não impede confirmação de uma agenda sobrecarregada.

Decisão ainda proposta. Investigar alertas de excesso de capacidade sem recusar serviços automaticamente. O módulo ServicoPlanejamento contém métodos provisórios que informam que a capacidade não foi implementada e retornam ausência de sugestão de prazo. O cálculo de carga e as transições de estado só poderão ser acrescentados após validação.

O que melhora	Trade-off / o que piora	O que fica mais difícil de mudar
Evita decisões automáticas incorretas e atende ao problema de visibilidade.	Depende de atenção do usuário; não garante agenda sem conflito.	Automatizar bloqueios depois exigirá revisar as regras, a interface e testes.

Evidência e próximo passo. tests/test_planejamento.py confirma apenas que nenhuma capacidade ou prazo é inventado. Ainda é necessário registrar as regras do cliente e criar testes reais de alerta, conflito e transição quando o comportamento for aprovado.

4. Módulos, segredos e dependências — Template 15

Módulo	Responsabilidade	Segredo / interface	Dependências e requisito
interface_web	Views/forms, páginas HTML e navegação de cadastro/listagem (implementados).	Views inicio() e novo_atendimento(); composição em container.py; formulário NovoAtendimentoForm.	Chama ServicoAtendimentos; C01 ainda sem teste cronometrado.
atendimentos	Cadastro, validações básicas e listagem de atendimentos; transições futuras.	ServicoAtendimentos.registrar() e listar_por_prioridade(); atualizar_estado() NÃO existe.	Depende da porta de repositório e da Strategy; H01-H03 atuais, H05 futura.
priorizacao	Ordenação por níveis manuais provisórios; cálculo por risco/tempo futuro.	PoliticaPrioridade.chave_ordenacao(); PrioridadeManual; ServicoDePriorizacao.ordenar().	Sem dependência de Django/ORM; RN01 parcialmente atendido, C02 não medido.

Módulo	Responsabilidade	Segredo / interface	Dependências e requisito
planejamento	Reserva de módulo para capacidade, alertas e prazos; SEM lógica funcional.	ServicoPlanejamento.avaliar_capacidade() retorna não implementado; sugerir_prazo() retorna None.	RN02/RN03/RN05 pendentes; não chamada pelo cadastro atual.
persistencia	Gravar/listar atendimentos fictícios via Django ORM (implementado).	DjangoAtendimentoAdapter.salvar()/listar(); porta em apps/atendimentos/ports.py.	Models, migração e SQLite; teste de integração publicado.

4.1 Regras de dependência

Regra	Justificativa	Verificação
Prioridade não importa views, templates ou models Django.	Manter regras testáveis e desacopladas.	Revisão de imports e testes unitários.
Views não consultam tabelas de prioridades diretamente.	Evitar duplicação de regra em interface.	Inspeção de views e testes de serviço.
Planejamento não altera estados sem passar pelo caso de uso.	Preservar consistência do fluxo.	Teste de transição e inspeção de chamadas.
Domínio depende de contrato de repositório, não de SQL específico.	Conter impacto da persistência.	Substituir implementação por repositório em memória no teste.
Não há chamadas em ciclo entre atendimentos, prioridades e planejamento.	Manter baixo acoplamento.	Análise de importações e organização de pacotes.

4.2 Acoplamentos aceitos e pontos sensíveis

Dependências observadas no código: interface_web → ServicoAtendimentos → PoliticaPrioridade e RepositorioAtendimento; o adaptador de persistência usa o Django ORM. Planejamento é pacote existente, mas não participa do fluxo de cadastro. A arquitetura mantém separação entre regra de domínio e armazenamento; metas de qualidade ainda precisam de medição.

Contratos efetivos: apps.priorizacao.policies.PoliticaPrioridade.chave_ordenacao(item); apps.atendimentos.ports.RepositorioAtendimento.salvar(atendimento) e listar(); apps.persistencia.adapters.DjangoAtendimentoAdapter. Não existem operações obter(id), listar(filtros) ou atualizar_estado() neste incremento.

5. Fichas de padrões de projeto — Template 16

P01 — Strategy (comportamental) para prioridade

Campo	Preenchimento
Problema	Critérios de prioridade podem variar; hoje não há pesos nem desempates validados. Referência: RN01 / C02.

Campo	Preenchimento
Justificativa	Isolar o algoritmo ou decisão de classificação do restante do sistema e permitir políticas substituíveis.
Alternativa sem padrão	Condicionais embutidas nas views ou serviço de atendimento; mudanças propagam para outras camadas.
Custo	Mais contrato, classes e indirection; possível excesso para apenas uma política.
Participantes	PoliticaPrioridade (Protocol), PrioridadeManual e ServicoDePriorizacao implementados. PrioridadePorRiscoETempo é somente possibilidade futura, ainda sem código/regra validada.
Teste / estado	IMPLEMENTADO parcialmente: tests/test_priorizacao.py contém dois testes aprovados (ordenação e desempate). Troca entre duas estratégias e alteração de prioridade não foram testadas.

P02 — Adapter (estrutural) para persistência

Campo	Preenchimento
Problema	Casos de uso devem ser testáveis sem banco real; pode existir troca futura de mecanismo de persistência. Referências: C02 / ADR-005.
Justificativa	Adaptar operações do Django ORM ao contrato RepositorioAtendimento consumido pelos casos de uso.
Alternativa sem padrão	Usar Model.objects diretamente em várias regras de negócio, com maior dependência do framework.
Custo	Classes e mapeamentos adicionais em um protótipo de pequeno porte.
Participantes	RepositorioAtendimento (porta em apps/atendimentos/ports.py); DjangoAtendimentoAdapter (apps/persistencia/adapters.py); AtendimentoModel (ORM); ServicoAtendimentos (cliente).
Teste / estado	IMPLEMENTADO: há repositório falso nos testes de serviço e teste de integração com ORM/Adapter; evidência geral: 10 testes executados com OK. Não há prova de testes de outros bancos.

Observação: não foi selecionado o padrão State. Para cinco estados simples, uma tabela de transições validada pode ser mais transparente; reavaliar apenas se os comportamentos por estado se tornarem complexos. Padrões não são um fim em si mesmos.

6. Registro de reúso — Template 17

Item / versão planejada	Finalidade e licença	Alternativas / motivo / ADR
Python — versão local não documentada	Linguagem e biblioteca padrão; PSF License.	Alternativas: Java / JavaScript; aprendizado e integração com Django. ADR-001.
Django==6.1.3 (fixado em requirements.txt)	Framework web, ORM, templates, forms, migrations, TestCase; BSD-3-Clause.	Alternativas: Spring Boot / Express; menor montagem inicial de infraestrutura. ADR-001/003/005.
SQLite (versão de runtime não documentada)	Banco SQL embarcado; núcleo em domínio público.	Alternativa: PostgreSQL; banco local para protótipo. ADR-005.
Django Templates / ORM / TestCase (utilizados)	Renderização HTML, persistência e testes; abrangidos pelo Django.	Evita dependências externas adicionais. ADR-001/003/005.
Git / GitHub (configurado; repositório público)	Histórico de versões, evidências e compartilhamento; serviço externo e ferramenta de desenvolvimento.	2 commits no ramo main, de Larissa Oliveira, em 12/11/2026; 0 pull requests. ADR-002 / gerência.

Fonte das versões: requirements.txt fixa Django==6.1.3; a configuração usa SQLite. A versão exata de Python e SQLite da máquina não está registrada. O código não identifica bibliotecas extras de terceiros além do Django na lista de dependências. O histórico Git público comprova os dois commits.

7. Esqueleto executável e evidências

Esqueleto executável entregue e versionado em https://github.com/lariolv25/gerenciamento_manutencao (ramo main). O repositório contém tela inicial, formulário de cadastro de atendimentos fictícios, persistência ORM/SQLite, ordenação por prioridade manual e suíte de 10 testes executados. Esta entrega NÃO implementa cálculo de risco/tempo, planejamento de capacidade, alertas de conflitos nem workflow completo de estados.

7.1 Organização sugerida de diretórios

```
gerenciamento_manutencao/
  manage.py                  # execução Django
  config/                    # settings / urls / wsgi / asgi
  apps/interface_web/        # views, forms, templates, composição
  apps/atendimentos/         # domínio, serviços e porta
  apps/priorizacao/          # Strategy manual
  apps/planejamento/         # métodos provisórios, sem cálculo
  apps/persistencia/         # ORM, migração e Adapter
  templates/                 # HTML
  static/                    # CSS
  docs/adr/                  # README sintético (ADRs individuais pendentes)
  docs/evidencias/           # resultado_testes.txt
  tests/                     # 10 testes de unidade e integração
  requirements.txt | README.md | .gitignore
```

7.2 Critérios para considerar o esqueleto entregue

- Critério ATENDIDO (relato de execução local): servidor Django inicia e apresenta a página inicial. O código inclui views, URLs e templates; há teste de integração da resposta HTTP 200.
- Critério ATENDIDO: a migração persistencia.0001_initial e o registro das apps estão no código; o log de testes mostra migrations aplicadas.

- Critério ATENDIDO: 10 testes executados com resultado final OK; log versionado em docs/evidencias/resultado_testes.txt.
- Critério ATENDIDO: README, .gitignore, requirements.txt e histórico Git (2 commits) estão públicos no ramo main.
- Critério PARCIAL: fronteiras de módulos implementadas; o documento contém os seis ADRs, mas docs/adr/ no repositório só inclui README sintético. ADRs individuais e ratificação formal pendentes.

Reprodução sugerida: criar ambiente virtual; instalar requirements.txt; executar python manage.py migrate, python manage.py check, python manage.py test tests -v 2 e python manage.py runserver. A evidência real arquivada mostra 10 testes aprovados; não há capturas de execução anexadas no repositório.

7.3 Verificações vinculadas aos cenários

Cenário / regra	Verificação planejada	Critério provisório
C01 — usabilidade	PENDENTE: nenhuma sessão móvel cronometrada foi anexada.	Meta proposta: até 60 s e até 5 passos; sem comprovação de cumprimento.
C02 — modificabilidade	PARCIAL: fronteiras e testes de unidade evidenciam separação; sem exercício cronometrado de mudança.	Meta proposta: até 4 h; não medida nem validada.
C03 — desempenho	PENDENTE: não há benchmark publicado para 100 atendimentos.	Meta proposta: até 2 s; não medida nem validada.
RN01 — priorização	EXECUTADO PARCIALMENTE: testes de ordenação e desempate da prioridade manual.	2 testes aprovados; pesos de risco/tempo e revisão de prioridade pendentes de validação.
RN04/RN05 — estados/capacidade	PENDENTE: sem transições ou alertas operacionais; há um teste de não implementação do planejamento.	Validar regras e implementar antes de exigir rejeição de transições ou alertas.

7.4 Rastreamento objetivo das evidências

Repositório: https://github.com/lariolv25/gerenciamento_manutencao (main). Histórico: /commits/main (2 commits). Saída integral de testes: docs/evidencias/resultado_testes.txt (10 testes; OK). Código: apps/priorizacao/policies.py (Strategy), apps/persistencia/adapters.py (Adapter), apps/atendimentos/ports.py (porta), apps/planejamento/services.py (funcionalidade não implementada). docs/adr/README.md é somente a síntese dos ADRs; os registros completos permanecem neste documento.

8. Gerência nesta etapa — Template 11

Período de referência: 10 a 15/11/2026; verificação do repositório em 12/11/2026. Projeto individual. Esta seção distingue itens comprovados por código e logs, tarefas apenas documentadas e validações externas ainda não demonstradas. Não foram informadas datas completas nem anexadas atas dos contatos desta etapa.

Planejado	Realizado até 12/11/2026	Evidência / pendência real
Revisar requisitos relevantes	Requisitos e hipóteses consolidados neste documento.	Registrar datas/atas de contatos e validação de regras ainda pendentes.

Planejado	Realizado até 12/11/2026	Evidência / pendência real
Comparar arquitetura e tecnologias	Seis ADRs discutidos; cinco materializados no esqueleto.	Ratificação formal não registrada; docs/adr/ contém só README.
Preparar esqueleto Django	Concluído: código Django/SQLite público, cadastro funcional e migrations.	GitHub: branch main; execução local relatada; 2 commits.
Produzir diagramas e contratos	Quatro figuras no relatório; contratos reais de Strategy e Adapter no código.	Diagramas de estados e capacidade permanecem propostas de evolução.
Planejar testes e incrementos	Concluída a suíte inicial: 10 testes aprovados e log publicado.	Métricas C01-C03 e regras H04-H08 ainda não validadas.

8.1 Riscos e acompanhamento

ID / situação	Risco e prioridade	Gatilho / mitigação
R01 / MITIGADO PARCIALMENTE	Curva de aprendizado Django/Python: risco operacional permanece, mas esqueleto e testes já executados.	Continuar testes e documentação; registrar versão do ambiente.
R02 / ATIVO	Escopo excessivo para execução individual; funcionalidades H04-H08 ainda não entregues.	Manter recorte incremental e validar prioridades do próximo ciclo.
R03 / ATIVO	Critérios automáticos por risco e tempo não confirmados pelo cliente.	Manter níveis manuais explicitamente provisórios; entrevistar cliente.
R04 / ATIVO	Estados, limite de horas/dia e política de conflitos não validados.	Não automatizar bloqueios; obter exemplos reais e testar depois.
R05 / MITIGADO PARCIALMENTE	Ausência inicial de evidências: já há 2 commits públicos e log dos 10 testes.	Manter novos commits e complementar com atas/prints quando disponíveis.

8.2 Retrospectiva e próximos ajustes

Retrospectiva verificada. O esqueleto Django foi versionado, o cadastro e a listagem por prioridade manual foram demonstrados e 10 testes passaram. A separação por módulos e os padrões Strategy/Adapter estão no código. Ainda precisam melhorar: histórico de validação com o cliente, formalização dos ADRs, métricas de usabilidade/modificabilidade/desempenho e regras de estados/capacidade. Próximo incremento: validar hipóteses antes de implementar cálculos ou bloqueios.

Gerência e rastreabilidade. Repositório público: https://github.com/lariolv25/gerenciamento_manutencao. Até 12/11/2026: 2 commits de Larissa Oliveira (esqueleto inicial e revisão da declaração de IA), 0 pull requests e 0 revisores; desenvolvimento individual direto em main. Quadro de tarefas externo: não documentado; atividades registradas na tabela desta seção. Atas/contatos: datas e anexos específicos não apresentados. Sessões planejadas: revisão do incremento entre 23 e 28/11/2026; aceitação até 12/12/2026 (não executadas). Pipeline CI: não configurado no repositório; considerar no próximo ciclo.

9. Declarações e responsabilidades — Templates 12 e 13

9.1 Declaração de contribuição individual

Etapa 2 — Projeto individual. Estudante: Larissa Kethen Gonçalves Oliveira | Matrícula: 22.2.8132. Escopo de atuação declarado: relação com o cliente, consolidação de requisitos, revisão das decisões arquiteturais, execução e avaliação local do esqueleto e dos testes, versionamento e documentação. O histórico público registra dois commits sob Larissa Oliveira; não há revisão por terceiros ou pull requests. Assinatura: Larissa Kethen Gonçalves Oliveira.

9.2 Declaração de utilização de IA generativa

Ferramenta: ChatGPT (OpenAI). Uso nesta etapa: auxílio na organização e redação do documento, comparação de alternativas, propostas de ADRs e diagramas, criação assistida do esqueleto Django, estrutura, formulários, regras provisórias e testes, além de instruções de execução. Verificações sob responsabilidade da estudante: avaliação crítica das soluções, execução local do servidor e dos testes, coleta das evidências e publicação dos commits no GitHub. A IA não realizou entrevistas reais, validações com o cliente, nem medições de usabilidade ou desempenho. O uso foi informado como autorizado pelo professor; compete à estudante observar as regras acadêmicas aplicáveis.

10. Pendências antes da submissão

Prioridade / situação	Pendência real e ação requerida
ALTA / PENDENTE	Confirmar/registrar datas e evidências dos contatos com o cliente na etapa; não inventar entrevistas ou atas.
ALTA / PENDENTE	Ratificar formalmente os ADRs 001–005; manter ADR-006 como hipótese. Publicar ADRs individuais se o professor exigir artefatos versionados.
ALTA / PENDENTE	Validar com cliente regras para estados, capacidade diária, conflitos, urgência e tempo antes de automatizar.
MÉDIA / PARCIAL	Conferir exemplos e dependências dos diagramas com a implementação; marcar explicitamente o que é arquitetura futura.
MÉDIA / PENDENTE	Registrar a versão exata de Python/SQLite do ambiente e, se necessário, capturas reais de servidor e testes.
MÉDIA / PENDENTE	Definir quadro de tarefas apenas se exigido; o relatório contém a tabela de acompanhamento, mas não há link/captura de quadro externo.
MÉDIA / PENDENTE	Cronometrar usabilidade; medir desempenho e modificabilidade em condições definidas no próximo incremento.
MÉDIA / PENDENTE	Conferir exigências finais no Moodle, assinatura da contribuição e eventuais anexos. Pipeline pode ficar para etapa seguinte.

Referências técnicas e acadêmicas

- UFOP / ICEA. CSI410 — Engenharia de Software II, Projeto de Curso (2026/2): organização e gerência, Etapa 1 e Templates 11, 14, 15, 16, 17, 12 e 13.
- Relatório anterior da estudante: Etapa 1 — Plano do Projeto, 09/11/2026; H01-H08, RN01-RN05 e C01-C03 (alguns itens preliminares).
- Django Software Foundation. FAQ de instalação do Django 6.1: https://docs.djangoproject.com/pt-br/6.1/faq/install/
- Django Software Foundation. Download e licença BSD: https://www.djangoproject.com/download/
- Django Software Foundation. Notas de versão 6.1: https://docs.djangoproject.com/en/6.1/releases/6.1/
- SQLite Consortium. Código do SQLite em domínio público: https://www.sqlite.org/copyright.html
- Python Software Foundation. Licença do Python: https://docs.python.org/3/license.html